

Universidad de Cundinamarca

Facultad de Ingeniería

Ingeniería de Software

Implementación de Autenticación con JWT en Aplicación Móvil SmartCampus

Documento Técnico — Actividad de Aprendizaje

Juan Francisco Mosquera Valero

Eider Medina Sepúlveda

Girardot, Cundinamarca

2026

Contenido

1. Introducción	3
2. Objetivos.....	3
Objetivo General.....	3
Objetivos Específicos.....	3
3. Arquitectura General del Sistema	3
4. Implementación de Autenticación con JWT	3
4.1 Envío de credenciales al backend	3
4.2 Generación del JWT en el backend	4
4.3 Recepción y almacenamiento del token.....	4
5. Gestión Global del Usuario con Zustand	4
5.1 Creación del store global.....	5
5.2 Acciones del store.....	5
5.3 Consulta del rol desde la base de datos	5
5.4 Persistencia del estado	6
6. Persistencia de Sesión	6
6.1 Estrategia utilizada.....	6
6.2 Recuperación automática del token	6
6.3 Validación de expiración y refresco automático.....	6
7. Control de Navegación y Guards	8
7.1 Protección de vistas restringidas	8
7.2 Guards de navegación	8
7.3 Acceso basado en roles	8
8. Integración con Backend Protegido.....	9
8.1 Envío del token en peticiones protegidas.....	9
8.2 Middleware de autenticación	9
8.3 Protección de endpoints	10
8.4 Endpoint de refresco de token en el backend	10
9. Flujo Completo de Autenticación.....	11
10. Buenas Prácticas de Seguridad	12
11. Resultados Obtenidos	12
12. Conclusiones.....	12
13. Referencias	12

1. Introducción

En el ecosistema de aplicaciones móviles modernas, la seguridad de los datos y la gestión eficiente de la identidad de los usuarios son pilares fundamentales. El proyecto SmartCampus surge como una solución integral para la gestión de espacios y servicios dentro de la Universidad de Cundinamarca, Girardot. Para garantizar que solo usuarios autorizados accedan a funcionalidades críticas, se ha implementado un robusto sistema de autenticación basado en JSON Web Tokens (JWT).

La utilización de JWT permite un intercambio de información seguro y compacto entre la aplicación móvil desarrollada en React Native y el backend construido con FastAPI. El backend y la base de datos se encuentran desplegados en Render, una plataforma de nube moderna que ofrece despliegue continuo y alta disponibilidad. Este documento detalla la implementación técnica de este flujo, desde el manejo de credenciales hasta la persistencia de sesión segura mediante Zustand y AsyncStorage.

2. Objetivos

Objetivo General

Desarrollar y documentar un sistema de autenticación seguro para la aplicación móvil SmartCampus utilizando estándares JWT, asegurando la integridad de las sesiones y el control de acceso basado en roles.

Objetivos Específicos

- Implementar el flujo de intercambio de credenciales y recepción de tokens en el frontend.
- Configurar un store global con Zustand para la gestión reactiva del estado de autenticación.
- Establecer guards de navegación para proteger rutas privadas según la identidad del usuario.
- Integrar un middleware de seguridad en el backend para validar la autenticidad de cada petición.

3. Arquitectura General del Sistema

La arquitectura de SmartCampus se basa en un modelo Cliente-Servidor desacoplado. El frontend, desarrollado con React Native y Expo, consume una API RESTful expuesta por un servidor FastAPI. Tanto el backend FastAPI como la base de datos PostgreSQL se encuentran alojados en Render, lo que permite un despliegue automático desde el repositorio y garantiza disponibilidad continua del servicio.

El flujo de autenticación sigue el estándar JWT: el usuario envía sus credenciales, el servidor las valida y retorna un Access Token y un Refresh Token. El cliente almacena estos tokens de forma persistente, permitiendo que las sesiones sobrevivan al cierre de la aplicación y se renueven automáticamente antes de expirar.

4. Implementación de Autenticación con JWT

4.1 Envío de credenciales al backend

El proceso inicia en la pantalla de Auth, donde el usuario ingresa su correo institucional y contraseña. La aplicación utiliza la API fetch para realizar una petición POST al endpoint /auth/login alojado en el servidor Render.

```
// Fragmento real de Auth.tsx
const res = await fetch(`${API_URL}/auth/login`, {
  method: 'POST',
  body: formData,
});
const data = await res.json();
if (!res.ok) {
  setError(data.detail || 'Correo o contraseña incorrectos.');
} else {
  setSuccessMessage('Iniciando sesion...');
  setTimeout(() => {
    login(data.user_id, data.token, data.refresh_token);
  }, 1000);
}
```

4.2 Generación del JWT en el backend

En el servidor FastAPI (desplegado en Render), se reciben las credenciales, se verifica el hash de la contraseña en la base de datos PostgreSQL y se genera el par de tokens necesarios firmados con el algoritmo HS256.

```
# Fragmento de routes_auth.py
@router.post('/login')
async def login(email: str = Form(...), password: str = Form(...)):
    row = await fetch_one(
        'SELECT id, password_hash FROM users WHERE email = $1', ...
    )
    if not row or not verify_password(password, row['password_hash']):
        raise HTTPException(401, 'Correo o contraseña incorrectos')

    access_token = create_access_token(data={'sub': str(row['id'])})
    refresh_token = create_refresh_token(data={'sub': str(row['id'])})
    return {
        'user_id': str(row['id']),
        'token': access_token,
        'refresh_token': refresh_token
    }
```

4.3 Recepción y almacenamiento del token

Una vez que el backend retorna una respuesta exitosa (200 OK), el frontend extrae el Access Token y el Refresh Token. Estos se pasan a la acción login del store global, la cual actualiza el estado reactivo y, gracias al middleware de persistencia, los almacena de forma segura en el almacenamiento local del dispositivo.

5. Gestión Global del Usuario con Zustand

Zustand se utiliza como la biblioteca de gestión de estado debido a su ligereza y facilidad de integración con React Native. El store centraliza toda la información del usuario y las acciones de autenticación.

5.1 Creación del store global

El store define la interfaz `AuthState`, que incluye el token, la información del perfil del usuario (nombre, rol, carrera) y el estado de autenticación.

```
// Fragmento de useAuthStore.ts
interface AuthState {
  token: string | null;
  refreshToken: string | null;
  userId: string | null;
  userName: string;
  userRole: UserRole;
  isAuthenticated: boolean;
  login: (userId: string, token: string, refreshToken?: string) => void;
  logout: () => void;
  fetchProfile: () => Promise<void>;
  refreshSession: () => Promise<boolean>;
}
```

5.2 Acciones del store

Las acciones `login` y `logout` permiten modificar el estado de forma atómica. La función `login` no solo guarda los tokens, sino que también dispara la carga automática del perfil del usuario mediante `fetchProfile()`.

```
login: (userId: string, token: string, refreshToken?: string) => {
  set({ userId, token, refreshToken: refreshToken || null, isAuthenticated: true });
  get().fetchProfile();
},
logout: () => {
  set({
    token: null,
    refreshToken: null,
    userId: null,
    isAuthenticated: false,
    userName: 'Usuario',
  });
},
```

5.3 Consulta del rol desde la base de datos

Una vez que el inicio de sesión es exitoso, se invoca automáticamente `fetchProfile()`, que realiza una petición GET al endpoint `/profiles/{user_id}` del backend en Render, enviando el Access Token en el header `Authorization`. El backend ejecuta una consulta SQL con JOIN para obtener el nombre completo y el rol del usuario:

```
-- Consulta ejecutada en el backend (routes_profiles.py)
SELECT u.full_name, r.role_name AS role
FROM users u
JOIN user_roles ur ON u.id = ur.user_id
JOIN roles r ON ur.role_id = r.id
WHERE u.id = $1
```

El resultado es retornado al frontend, donde `fetchProfile()` actualiza el store con el nombre y rol del usuario, habilitando el control de acceso basado en roles en la navegación:

```

fetchProfile: async () => {
  const { token, userId } = get();
  const res = await fetch(`${API_URL}/profiles/${userId}`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  if (res.ok) {
    const profile = await res.json();
    set({ userName: profile.full_name, userRole: profile.role });
  }
},

```

5.4 Persistencia del estado

Para que la sesión no se pierda al cerrar la aplicación, se utiliza el middleware persist de Zustand configurado con AsyncStorage.

```

persist(
  (set, get) => ({ ... }),
  {
    name: 'auth-storage',
    storage: createJSONStorage(() => AsyncStorage),
  }
)

```

6. Persistencia de Sesión

6.1 Estrategia utilizada

Se emplea una estrategia de rehidratación de estado. Al iniciar la aplicación, Zustand recupera los datos desde AsyncStorage y restaura el estado isAuthenticated, permitiendo al usuario continuar su sesión sin necesidad de volver a autenticarse.

6.2 Recuperación automática del token

El hook useAuthStore verifica si existe un token persistido al arrancar la aplicación. Si el token es válido, el sistema permite el acceso directo a las pantallas protegidas sin pasar por la pantalla de autenticación.

6.3 Validación de expiración y refresco automático

El sistema implementa un mecanismo de refresco automático de sesión que opera de forma transparente para el usuario. Se creó un helper centralizado denominado authenticatedFetch() que actúa como interceptor de todas las peticiones autenticadas: detecta respuestas con código 401 (Unauthorized), intenta renovar el Access Token usando el Refresh Token y reintenta la petición original de forma automática, sin interrumpir la experiencia del usuario.

```

// utils/authenticatedFetch.ts
// Interceptor centralizado - se usa en todos los componentes
// en lugar de llamar a fetch() directamente.
export async function authenticatedFetch(
  url: string,
  options: RequestInit = {}
): Promise<Response> {

```

```

const { token, refreshSession, logout } = useAuthStore.getState();

// 1. Adjunta el Access Token en el header de la petición
const authOptions = {
  ...options,
  headers: {
    ...options.headers,
    Authorization: `Bearer ${token}`,
  },
};

let response = await fetch(url, authOptions);

// 2. Detecta el error 401 (token expirado o invalido)
if (response.status === 401) {
  // 3. Intenta renovar el Access Token con el Refresh Token
  const refreshed = await refreshSession();

  if (refreshed) {
    // 4. Reintenta la petición con el nuevo Access Token
    const newToken = useAuthStore.getState().token;
    response = await fetch(url, {
      ...options,
      headers: {
        ...options.headers,
        Authorization: `Bearer ${newToken}`,
      },
    });
  } else {
    // 5. Refresh Token expirado: cierra sesión automáticamente
    logout();
  }
}

return response;
}

```

El método `refreshSession()` reside en el store de Zustand (`useAuthStore.ts`) y es invocado exclusivamente a través de este interceptor, garantizando un único punto de control para la renovación de tokens:

```

// useAuthStore.ts - acción refreshSession
refreshSession: async () => {
  const { refreshToken } = get();
  if (!refreshToken) return false;

  const formData = new FormData();
  formData.append('refresh_token', refreshToken);

  const res = await fetch(`${API_URL}/auth/refresh`, {
    method: 'POST',
    body: formData,
  });

  if (res.ok) {
    const data = await res.json();
    set({ token: data.token }); // Actualiza el store con el nuevo Access Token
    return true;
  }
  return false;
},

```

Uso en componentes: cualquier pantalla o servicio que necesite realizar una petición autenticada utiliza `authenticatedFetch()` en lugar de `fetch()` directamente, garantizando que el refresco automático ocurra de forma uniforme en toda la aplicación:

```
// screens/ReservationsScreen.tsx
const loadReservations = async () => {
  // Se usa authenticatedFetch en lugar de fetch() directo
  const res = await authenticatedFetch(`${API_URL}/reservations`);

  if (res.ok) {
    const data = await res.json();
    setReservations(data);
  }
  // Si el token expiro, authenticatedFetch lo renueva automaticamente.
  // Si el Refresh Token expiro, cierra la sesion y redirige a Auth.
};
```

7. Control de Navegación y Guards

La seguridad en el frontend se maneja mediante el control de la pila de navegación (Stack) basado en el estado reactivo del usuario.

7.1 Protección de vistas restringidas

En el archivo raíz `_layout.tsx` se evalúa la propiedad `isAuthenticated` del store. Dependiendo de este valor, el usuario es dirigido al flujo de autenticación (Intro/Auth) o al contenido principal de la aplicación.

```
// Fragmento de _layout.tsx
{!isAuthenticated ? (
  !isIntroFinished ? (
    <Intro onFinish={handleIntroFinish} />
  ) : (
    <Auth />
  )
) : (
  <Stack screenOptions={{ headerShown: false }}>
    <Stack.Screen name='(tabs)' options={{ headerShown: false }} />
    { /* Pantallas protegidas */ }
  </Stack>
)}
```

7.2 Guards de navegación

El guard es intrínseco al renderizado condicional. Si un usuario no autenticado intenta acceder a una ruta interna, el sistema lo redirecciona automáticamente a la pantalla de autenticación, sin necesidad de un guard explícito separado.

7.3 Acceso basado en roles

Además de la autenticación, se valida el rol del usuario (admin, student, teacher) para habilitar o deshabilitar pestañas específicas en la navegación inferior. El rol es cargado desde la base de datos durante `fetchProfile()` y almacenado en el store.


```
// Fragmento de (tabs)/_layout.tsx
<Tabs.Screen
  name='requests'
  options={{
    title: 'Solicitudes',
    // href: undefined => visible; href: null => oculto
    href: role === 'admin' ? undefined : null,
    tabBarIcon: ({ color }) => (
      <Icons size={24} name='checkmark-circle-outline' color={color} />
    ),
  }}
/>
```

8. Integración con Backend Protegido

El backend FastAPI está desplegado en Render, que gestiona automáticamente el ciclo de vida del servidor, las variables de entorno (SECRET_KEY, REFRESH_SECRET_KEY, DATABASE_URL) y el despliegue continuo desde el repositorio.

8.1 Envío del token en peticiones protegidas

Para cada petición a un endpoint protegido, la aplicación adjunta el Access Token en el header Authorization siguiendo el esquema Bearer. Esto garantiza que el servidor pueda verificar la identidad del usuario en cada solicitud.

```
// Ejemplo de petición autenticada desde el frontend
const { token } = useAuthStore.getState();

const res = await fetch(`${API_URL}/reservations`, {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json',
    Authorization: `Bearer ${token}`,
  },
});
```

8.2 Middleware de autenticación

El backend utiliza un sistema de dependencias de FastAPI para proteger los recursos. La función `verify_token` actúa como middleware: decodifica el JWT, valida su firma y verifica si el usuario se encuentra activo en la base de datos PostgreSQL alojada en Render.

```
# Fragmento de auth_utils.py
async def verify_token(
    credentials: HTTPAuthorizationCredentials = Depends(security)
):
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id = payload.get('sub')
        # Verifica si el usuario existe y esta activo en DB
        row = await fetch_one(
            'SELECT active FROM users WHERE id = $1', ..., user_id
        )
        if not row['active']:
            raise HTTPException(status_code=403, detail='Cuenta inactiva')
        return payload
```

```
except jwt.ExpiredSignatureError:
    raise HTTPException(401, 'Token expirado')
```

8.3 Protección de endpoints

Cualquier endpoint que requiera protección simplemente declara una dependencia hacia `verify_token`. De esta forma, FastAPI verifica el token antes de ejecutar la lógica del endpoint.

```
# Fragmento de routes_profiles.py
@router.get('/{user_id}')
async def get_profile(
    user_id: int,
    token_payload: dict = Depends(verify_token)
):
    # Solo accesible si el token es valido
    row = await fetch_one(query, ..., user_id)
    return row
```

8.4 Endpoint de refresco de token en el backend

El backend expone el endpoint `/auth/refresh` que recibe el Refresh Token, valida su firma y expiración, y en caso de ser válido genera un nuevo Access Token sin necesidad de que el usuario vuelva a iniciar sesión.

```
# Fragmento de routes_auth.py
@router.post('/refresh')
async def refresh_token(refresh_token: str = Form(...)):
    try:
        # Valida firma y expiracion del Refresh Token
        payload = jwt.decode(
            refresh_token, REFRESH_SECRET_KEY, algorithms=[ALGORITHM]
        )
        user_id = payload.get('sub')
        if not user_id:
            raise HTTPException(401, 'Token invalido')

        # Verifica que el usuario sigue activo en DB
        row = await fetch_one(
            'SELECT active FROM users WHERE id = $1', ..., user_id
        )
        if not row or not row['active']:
            raise HTTPException(403, 'Cuenta inactiva')

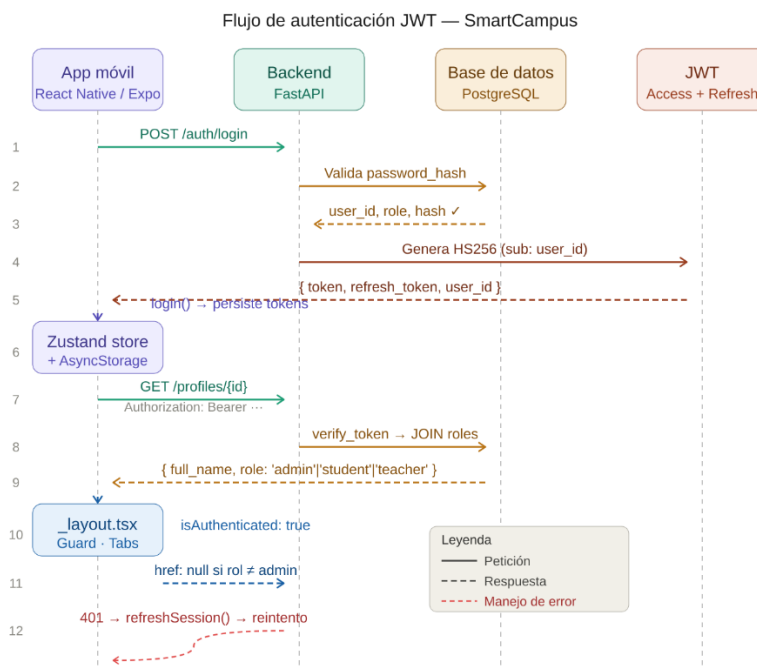
        # Genera nuevo Access Token (duracion: 30 minutos)
        new_access_token = create_access_token(
            data={'sub': user_id},
            expires_delta=timedelta(minutes=30)
        )
        return {'token': new_access_token}

    except jwt.ExpiredSignatureError:
        # Refresh Token expirado: el usuario debe iniciar sesion nuevamente
        raise HTTPException(401, 'Refresh token expirado.')
    except jwt.JWTError:
        raise HTTPException(401, 'Refresh token invalido')
```

Los tiempos de expiración configurados son: Access Token con duración de 30 minutos para minimizar el riesgo ante filtraciones, y Refresh Token con duración de 7 días para garantizar una experiencia de usuario fluida sin reautenticaciones frecuentes.

9. Flujo Completo de Autenticación

1. El usuario ingresa sus credenciales en la aplicación móvil.
2. La aplicación envía una petición POST /auth/login al backend FastAPI en Render.
3. El servidor FastAPI valida las credenciales contra la base de datos PostgreSQL en Render.
4. El servidor genera un par de tokens JWT (Access y Refresh) firmados con HS256.
5. La aplicación recibe los tokens y actualiza el store de Zustand.
6. El middleware persist de Zustand guarda los tokens en AsyncStorage.
7. El estado isAuthenticated cambia a true, lo que activa el guard de navegación.
8. La aplicación invoca fetchProfile() enviando el token vía Authorization: Bearer para cargar el nombre y rol del usuario desde la base de datos.
9. El usuario es redirigido automáticamente al Dashboard (tabs) con su rol asignado.
10. Para cada petición protegida, authenticatedFetch() adjunta el Access Token en el header Authorization: Bearer.
11. Si el token expira (error 401), authenticatedFetch() invoca refreshSession(), que llama a /auth/refresh en Render para obtener un nuevo Access Token sin cerrar la sesión.
12. Si el Refresh Token también expiró, se ejecuta logout() y el usuario es redirigido a la pantalla de autenticación.



10. Buenas Prácticas de Seguridad

- **Hashing de contraseñas:** Se utiliza pbkdf2_sha256 para evitar el almacenamiento de claves en texto plano.
- **Tokens de duración limitada:** El Access Token tiene validez de 30 minutos y el Refresh Token de 7 días, reduciendo la ventana de riesgo ante posibles filtraciones.
- **Almacenamiento seguro:** Uso de AsyncStorage con el middleware persist de Zustand para mantener la sesión de forma eficiente entre sesiones.
- **Principio de Mínimo Privilegio:** El acceso a los módulos se restringe según el rol del usuario autenticado (admin, student, teacher).
- **Transmisión segura del token:** El Access Token se transmite únicamente vía el header HTTP Authorization: Bearer, nunca en la URL ni en el cuerpo de la petición.
- **Variables de entorno en Render:** Las variables sensibles como SECRET_KEY, REFRESH_SECRET_KEY y DATABASE_URL se gestionan como variables de entorno en Render, sin estar expuestas en el código fuente.
- **Interceptor centralizado:** Un único interceptor centralizado (authenticatedFetch) gestiona la detección del error 401 y el refresco del token, evitando lógica duplicada en los componentes.

11. Resultados Obtenidos

Se ha logrado una integración exitosa entre el frontend móvil y el backend desplegado en Render. Los usuarios pueden registrarse, iniciar sesión y navegar por la aplicación manteniendo su sesión activa incluso tras reiniciar el dispositivo. El rol del usuario se consulta y carga automáticamente desde la base de datos PostgreSQL al momento del inicio de sesión, habilitando el control de acceso diferenciado por pestañas en la navegación. El mecanismo de refresco automático de token, implementado a través del interceptor authenticatedFetch(), funciona de forma transparente sin interrumpir la experiencia del usuario. Cuando el Refresh Token expira, la sesión se cierra de forma controlada y el usuario es redirigido a la pantalla de autenticación.

12. Conclusiones

La implementación de un sistema de autenticación JWT en el proyecto SmartCampus proporciona una base sólida de seguridad y escalabilidad. La combinación de FastAPI como backend de alto rendimiento desplegado en Render, PostgreSQL como base de datos persistente, y Zustand como gestor de estado simplifica el flujo de datos y mejora la experiencia del usuario. La consulta del rol directamente desde la base de datos mediante una consulta SQL con JOIN garantiza que los permisos reflejen siempre el estado real del sistema. El interceptor centralizado authenticatedFetch() asegura que el refresco automático de tokens ocurra de forma uniforme en toda la aplicación, sin lógica duplicada. Este desarrollo cumple con los estándares de la industria para aplicaciones móviles modernas y garantiza la protección de la infraestructura universitaria.

13. Referencias

FastAPI. (2026). Security - OAuth2 with Password (and hashing), Bearer with JWT tokens. <https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>

React Native. (2026). AsyncStorage documentation. <https://react-native-asyncstorage.github.io/async-storage/>

Zustand. (2026). Persistence middleware documentation. <https://docs.pmnd.rs/zustand/integrations/persisting-store-data>

JSON Web Token. (2026). Introduction to JSON Web Tokens. <https://jwt.io/introduction/>

Render. (2026). Deploy a FastAPI App. <https://render.com/docs/deploy-fastapi>